

VITyarthi | Build Your Own Project

# Movie Ticket Booking System

Project Report

|                  |                                                      |
|------------------|------------------------------------------------------|
| Submitted by     | Atharv Maldhure                                      |
| Registration No. | 26BAI10615                                           |
| Programme        | B.Tech Computer Science and Engineering (First Year) |
| Project type     | Build Your Own Project (individual)                  |
| Language         | Python 3                                             |
| Submission       | September 2026                                       |

# Table of Contents

- 1. Introduction . . . . . 3
- 2. Problem Statement . . . . . 3
- 3. Functional Requirements . . . . . 4
- 4. Non-Functional Requirements . . . . . 5
- 5. System Architecture . . . . . 6
- 6. Design Diagrams . . . . . 7
  - 6.1 Use Case Diagram . . . . . 7
  - 6.2 Workflow Diagrams . . . . . 8
  - 6.3 Sequence Diagram . . . . . 10
  - 6.4 Class / Component Diagram . . . . . 11
  - 6.5 ER Diagram and Data Model . . . . . 11
- 7. Design Decisions and Rationale . . . . . 12
- 8. Implementation Details . . . . . 12
- 9. Screenshots and Results . . . . . 14
- 10. Testing Approach . . . . . 18
- 11. Challenges Faced . . . . . 19
- 12. Learnings and Key Takeaways . . . . . 19
- 13. Future Enhancements . . . . . 20
- 14. References . . . . . 20
- Appendix A: Source Code (main.py) . . . . . 21

# 1. Introduction

## 1.1 Background

Booking a cinema ticket usually means standing in a queue, asking about movies and show timings, and calculating the price by hand. This is slow for the customer and error-prone for the counter staff. A simple software system can show the available movies and timings, calculate the total price automatically, and keep a record of every booking.

## 1.2 Project Overview

The **Movie Ticket Booking System** is a menu-driven, command-line application written in Python. After entering their name, a user can view the movies currently showing with their ticket prices, view the show timings, book tickets for a chosen movie and show, and review all bookings made during the session. Every input is validated so that mistakes such as typing text instead of a number, choosing a movie that does not exist, or entering zero tickets are reported clearly and never crash the program.

## 1.3 Tools and Technologies

- **Language:** Python 3 (uses f-strings, so Python 3.6 or higher is required)
- **Libraries:** none. Only built-in Python features are used (dictionaries, lists, functions, exceptions)
- **Interface:** console (text menu) using `input()` and `print()`
- **Version control:** Git and GitHub

# 2. Problem Statement

## 2.1 Problem Statement

Manual ticket booking is time-consuming and depends on the staff member correctly noting the movie, the show time, the number of tickets and the total price. Customers cannot easily see all movies and timings at once, and there is no simple record of what has been booked. The problem is to design and build a small software system that lets a user browse movies and show timings, book tickets with automatic price calculation, and view their bookings, while safely handling invalid input.

## 2.2 Objectives

- Display the list of available movies with ticket prices and the available show timings
- Allow a user to book tickets by selecting a movie, a show time and the number of tickets
- Calculate the total amount automatically and ask for confirmation before saving a booking
- Let the user view all confirmed bookings made during the session
- Validate every input so that wrong or unexpected input never crashes the program
- Apply core programming concepts: functions, dictionaries, lists, loops, conditionals and exception handling

## 2.3 Scope

**In scope:** a console application with four movies and four show times, ticket booking with price calculation, confirmation, booking history for the running session, and input validation.

**Out of scope (in this version):** seat selection, online payment, user accounts or passwords, booking cancellation after confirmation, and permanent storage in a file or database.

## 2.4 Target Users

- **Cinema-goers** who want to book tickets quickly without confusion
- **Counter staff** who can use the program to speed up manual billing
- **Students** learning Python, who can use the project as a simple example of a menu-driven program

## 3. Functional Requirements

The system is organised into three major functional modules, supported by a main menu and an input-validation helper.

| ID   | Module            | Requirement                                                                                                                         |
|------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| FR1  | Movie Information | Display all available movies with their ticket price.                                                                               |
| FR2  | Movie Information | Display all available show timings.                                                                                                 |
| FR3  | Ticket Booking    | Let the user choose a movie by its number.                                                                                          |
| FR4  | Ticket Booking    | Let the user choose a show time by its number.                                                                                      |
| FR5  | Ticket Booking    | Let the user enter the number of tickets (must be a whole number greater than zero).                                                |
| FR6  | Ticket Booking    | Calculate the total amount as number of tickets multiplied by the ticket price.                                                     |
| FR7  | Ticket Booking    | Show a booking summary (movie, time, tickets, total) and ask the user to confirm (yes / no).                                        |
| FR8  | Ticket Booking    | Save the booking only when the user confirms; otherwise cancel it.                                                                  |
| FR9  | Booking History   | Display all confirmed bookings of the session, numbered, with movie, time, tickets and total.                                       |
| FR10 | Booking History   | Display a message when no bookings exist.                                                                                           |
| FR11 | Main Menu         | Ask for the user's name, show a five-option menu repeatedly, and end with a personalised goodbye message on Exit.                   |
| FR12 | Validation        | Reject non-numeric input, unavailable movie or timing numbers, invalid ticket counts and invalid menu choices with a clear message. |

### 3.1 Input and Output Structure

| Input             | Type / allowed values          | Output produced                                                 |
|-------------------|--------------------------------|-----------------------------------------------------------------|
| User name         | Any text                       | Used in the goodbye message                                     |
| Menu choice       | 1 to 5                         | Movie list, timing list, booking flow, booking history, or exit |
| Movie number      | Integer 1 to 4                 | Selected movie and price, or 'Movie not available.'             |
| Timing number     | Integer 1 to 4                 | Selected show time, or 'Timing not available.'                  |
| Number of tickets | Integer greater than 0         | Booking summary with total, or an error message                 |
| Confirmation      | 'yes' (any other text cancels) | 'Booking successful!' or 'Booking cancelled.'                   |

### 3.2 User Workflow

- 1 The user starts the program and enters their name.
- 2 The main menu is displayed: Movies, Show Timings, Book Ticket, My Bookings, Exit.
- 3 The user may view movies and timings at any time (options 1 and 2).
- 4 To book, the user selects option 3, then a movie, a show time and the number of tickets.
- 5 The system shows the booking details and total, and asks for confirmation.
- 6 On 'yes' the booking is saved; on anything else it is cancelled. The menu is shown again.
- 7 The user may view all bookings (option 4) and finally exit (option 5).

## 4. Non-Functional Requirements

| ID   | Category                     | Requirement                                          | How it is achieved                                                                                                                                      |
|------|------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR1 | Usability                    | The program must be easy to use without training.    | Numbered menus, clear prompts such as 'Enter movie number:', and readable output with aligned labels.                                                   |
| NFR2 | Reliability / error handling | Invalid input must never crash the program.          | <code>readnumber()</code> catches <code>ValueError</code> ; calls <code>check</code> for unavailable choices and invalid counts and return to the menu. |
| NFR3 | Performance                  | Every action must respond instantly.                 | Data is held in memory in dictionaries, so looking up a movie or timing by number takes constant time.                                                  |
| NFR4 | Maintainability              | Changing movies, prices or timings should be easy.   | Data is kept in the constants <code>MOVIES</code> and <code>SHOW_TIMES</code> , separate from the logic; each function does one task.                   |
| NFR5 | Portability                  | The program must run on any common operating system. | Pure Python 3 with no external libraries or operating-system specific calls.                                                                            |
| NFR6 | Resource efficiency          | Memory and processing use must stay minimal.         | Only a small dictionary and a list of booking records are stored; no files, network or database are used.                                               |

### 4.1 Error Handling Strategy

All numeric input goes through one helper, `readnumber(prompt)`. It converts the text to an integer, and if conversion fails it prints 'Please enter a number.' and returns `None`. The calling function then treats `None` like any other invalid value: it is not a key of `MOVIES` or `SHOW_TIMES`, and it fails the check for the ticket count. Each failed check prints a specific message and returns to the main menu, so the user can simply try again.

## 5. System Architecture

The application follows a simple three-layer structure, all inside one Python program. The **presentation layer** is the console: the menu loop in `main()` and the calls to `input()` and `print()`. The **application logic layer** contains the functions that carry out each feature and validate input. The **data layer** holds the movie list, the show timings and the bookings in memory.

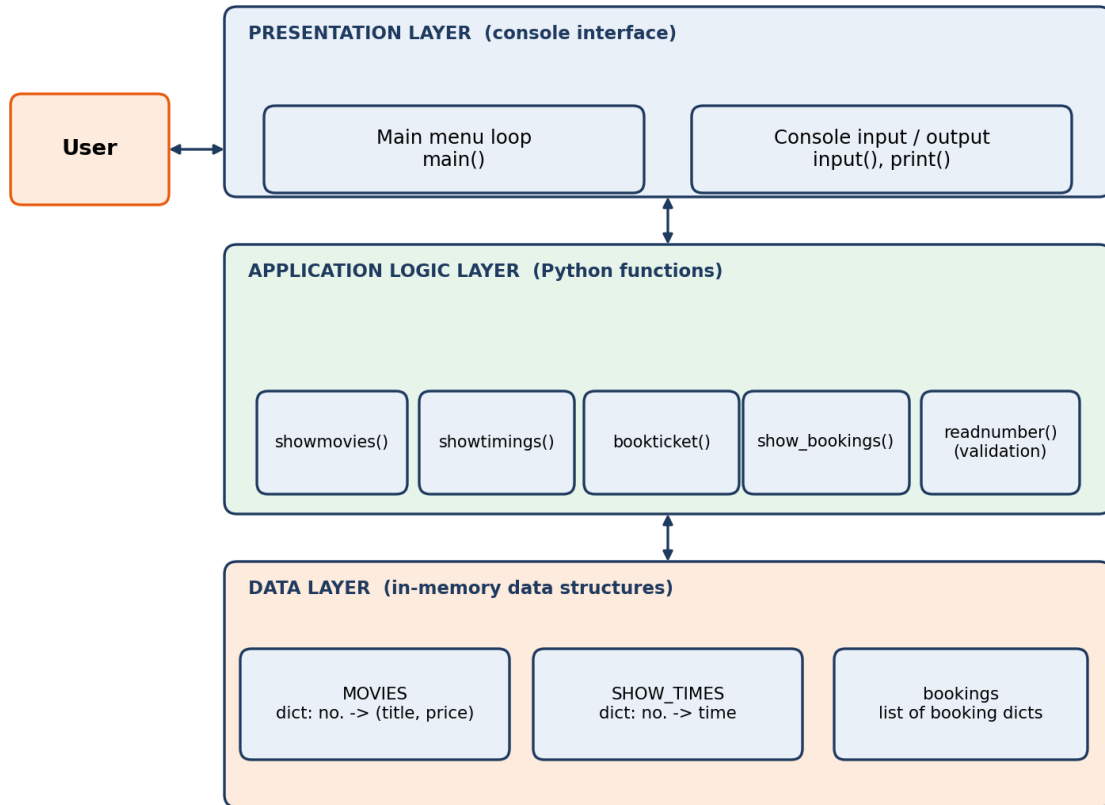


Figure 1: System architecture (three layers)

| Layer             | Components                                                                                                                                    | Responsibility                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Presentation      | <code>main()</code> , <code>input()</code> , <code>print()</code>                                                                             | Shows the menu, reads the user's choice and displays results and messages.                              |
| Application logic | <code>showmovies()</code> , <code>showtimings()</code> , <code>bookticket()</code> , <code>show_bookings()</code> , <code>readnumber()</code> | Implements each feature, calculates the total price, validates input and applies the confirmation step. |
| Data              | <code>MOVIES</code> , <code>SHOW_TIMES</code> , <code>bookings</code>                                                                         | Stores movie titles and prices, show timings and confirmed bookings while the program runs.             |

## 6. Design Diagrams

### 6.1 Use Case Diagram

The only actor is the **User**. Booking a ticket includes two supporting use cases: validating each input, and confirming the booking before it is saved.

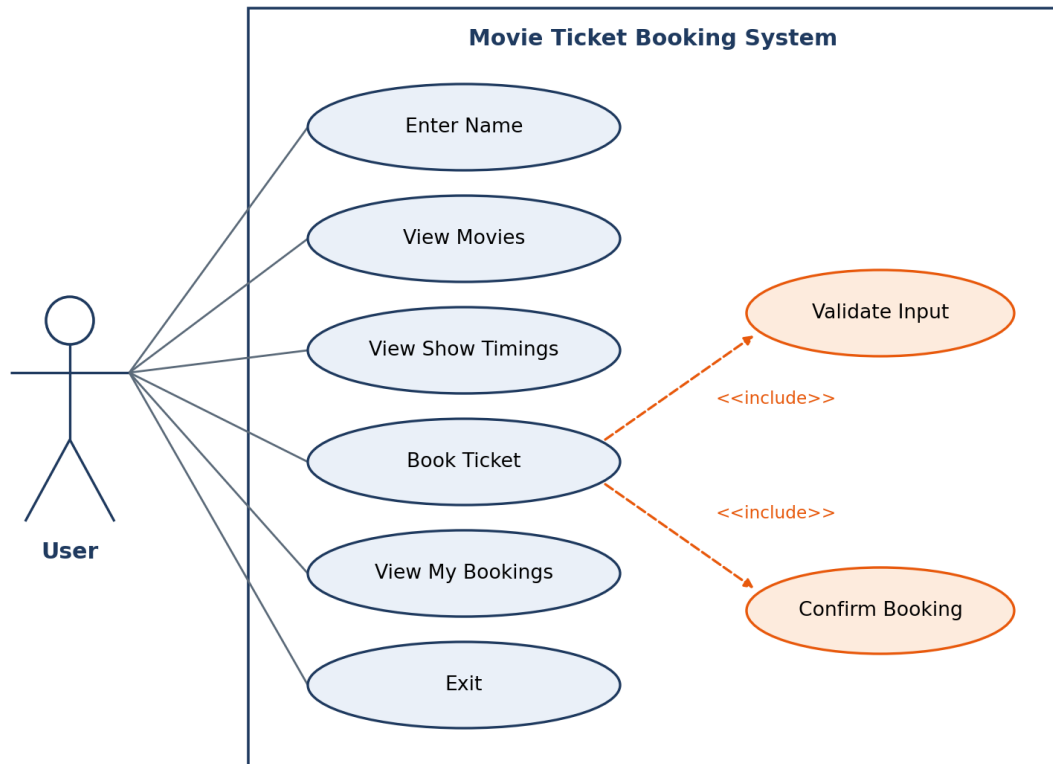


Figure 2: Use case diagram

| Use case                   | Description                                                                                           |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| Enter Name                 | The user gives a name at start-up; it is used in the goodbye message.                                 |
| View Movies                | Lists every movie with its ticket price.                                                              |
| View Show Timings          | Lists every available show time.                                                                      |
| Book Ticket                | Guides the user through choosing a movie, a show time and the number of tickets, and shows the total. |
| Validate Input (included)  | Checks each entry and rejects invalid values with a message.                                          |
| Confirm Booking (included) | Saves the booking only if the user answers 'yes'.                                                     |
| View My Bookings           | Shows all bookings confirmed in the current session.                                                  |
| Exit                       | Ends the program with a goodbye message.                                                              |

## 6.2 Workflow Diagrams

Figure 3 shows the overall program flow: the menu is displayed repeatedly until the user chooses Exit. Figure 4 shows the detailed flow inside the ticket-booking function, including every validation check.

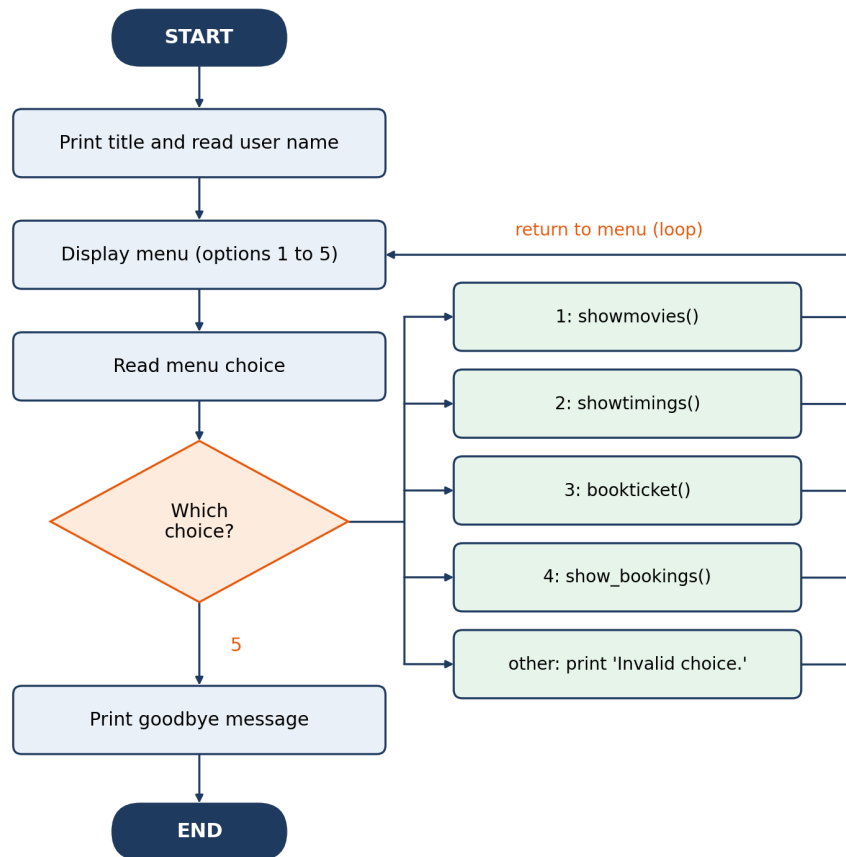


Figure 3: Workflow of the main menu loop

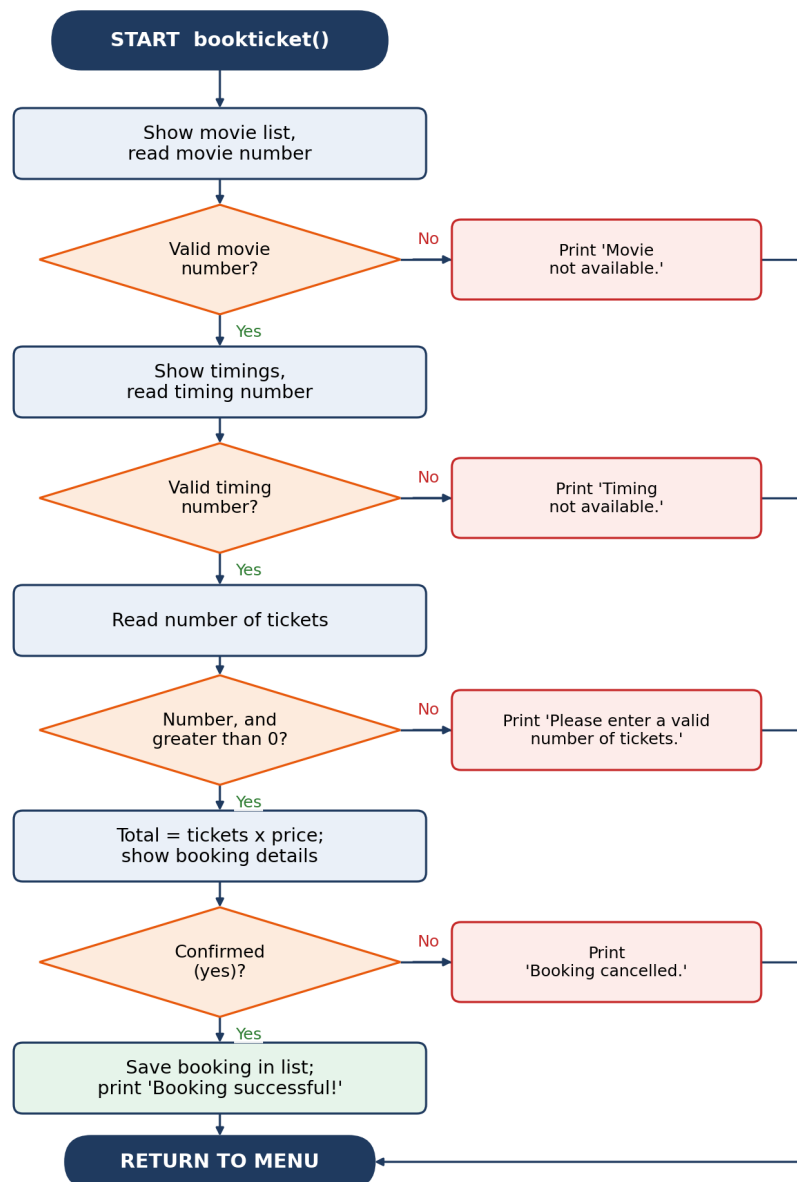


Figure 4: Workflow of the ticket booking process

## 6.3 Sequence Diagram

Figure 5 shows the interaction between the user and the program parts during a successful booking. Dashed arrows are returns or displayed output. The user's three numeric entries are read through `readnumber()`.

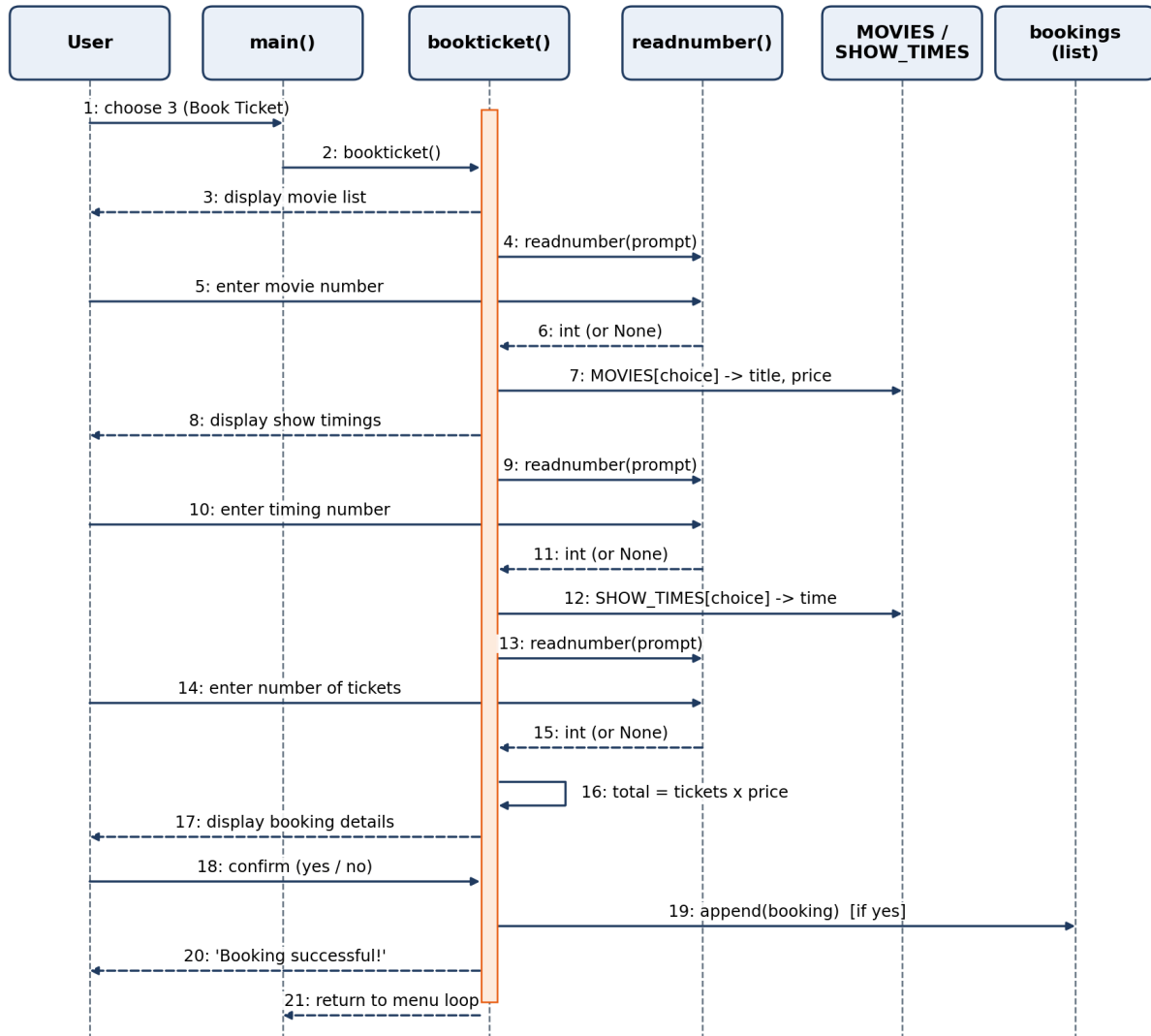


Figure 5: Sequence diagram of a successful ticket booking

## 6.4 Class / Component Diagram

The program is written using functions rather than classes, so a **component diagram** is used. `main()` calls the four feature functions, `bookticket()` uses `readnumber()` for validated input, and the functions read from or write to the shared data structures.

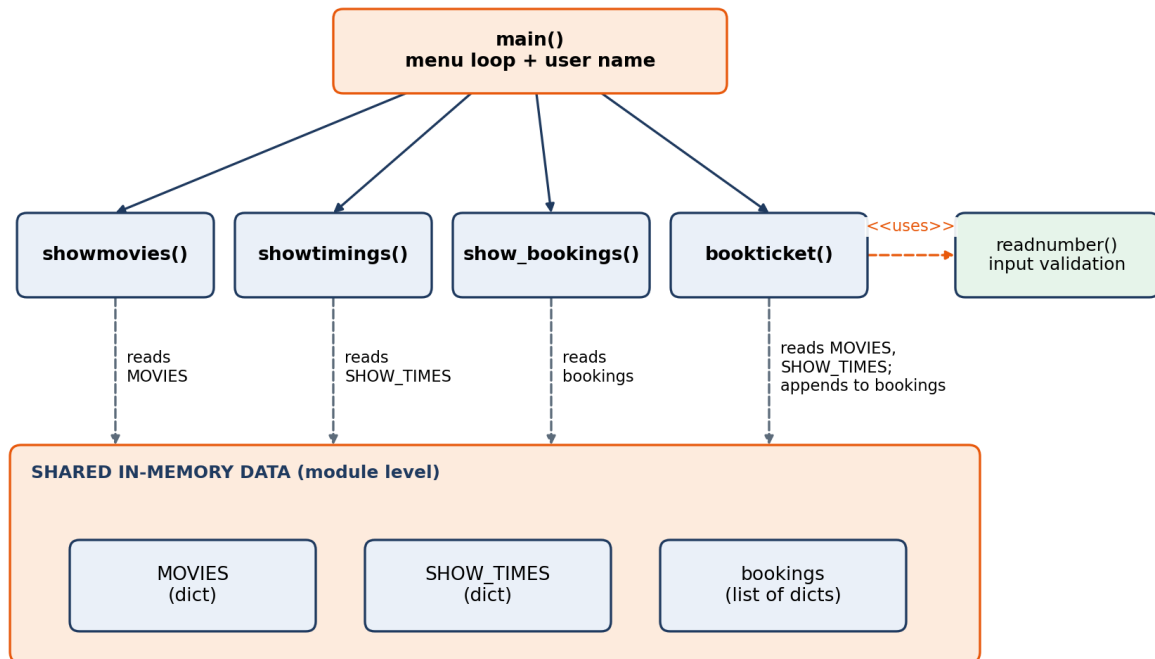


Figure 6: Component diagram

| Component                       | Responsibility                                                                                              |
|---------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>main()</code>             | Reads the user's name, displays the menu in a loop and calls the function for the selected option.          |
| <code>showmovies()</code>       | Prints each movie number, title and price from <code>MOVIES</code> .                                        |
| <code>showtimings()</code>      | Prints each show time number and time from <code>SHOW_TIMES</code> .                                        |
| <code>bookticket()</code>       | Runs the complete booking flow: selection, validation, total calculation, confirmation and saving.          |
| <code>show_bookings()</code>    | Prints all saved bookings, or a message if there are none.                                                  |
| <code>readnumber(prompt)</code> | Reads an integer safely; returns <code>None</code> (after printing a message) if the input is not a number. |

## 6.5 ER Diagram and Data Model

The system does **not** use a database or any file storage, so an ER diagram is not applicable. All data lives in three in-memory Python structures for the duration of the run. Their structure is documented below in place of an ER diagram.

| Structure           | Python type             | Content                                                                                            |
|---------------------|-------------------------|----------------------------------------------------------------------------------------------------|
| <code>MOVIES</code> | dict: int to (str, int) | Menu number mapped to a tuple of (movie title, ticket price in Rs.), e.g. 3: ('Interstellar', 220) |

| Structure  | Python type      | Content                                                                           |
|------------|------------------|-----------------------------------------------------------------------------------|
| SHOW_TIMES | dict: int to str | Menu number mapped to the show time text, e.g. 3: '6:00 PM'                       |
| bookings   | list of dict     | One dictionary per confirmed booking with the keys movie, time, tickets and total |

## 7. Design Decisions and Rationale

| Decision                                                     | Rationale                                                                                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Dictionaries for MOVIES and SHOW_TIMES, keyed by menu number | The number the user types is the key, so validation is one check (choice not in MOVIES) and lookup is direct.                   |
| Tuple (title, price) as the value in MOVIES                  | Title and price belong together and do not change, so an immutable tuple fits and can be unpacked in one line.                  |
| List of dictionaries for bookings                            | Bookings are added in order and displayed in order; named keys make the printing code easy to read.                             |
| One helper, readnumber(), for all numeric input              | Avoids repeating try/except code and gives consistent error messages.                                                           |
| Return None on bad numeric input                             | The caller decides what to do; a failed conversion is handled by the same 'not available / not valid' checks.                   |
| Early return after each failed check                         | Keeps bookticket() flat and readable instead of deeply nested if statements, and always returns the user to the menu.           |
| Confirmation step before saving                              | Lets the user review the total and cancel; only an explicit 'yes' creates a booking.                                            |
| Accept confirmation with strip() and lower()                 | ' YES ' and 'Yes' are accepted, so harmless differences in typing do not cancel a booking.                                      |
| Separate function per menu option                            | Each feature can be understood, changed and tested on its own; main() only routes choices.                                      |
| In-memory storage only                                       | Keeps the project simple and dependency-free for this course level; file or database storage is listed as a future enhancement. |
| Console (text) interface                                     | Focuses the project on logic and validation, and runs anywhere Python is installed.                                             |

## 8. Implementation Details

### 8.1 Development Environment

The project is a single Python source file, `main.py`, of about 100 lines. It needs Python 3.6 or higher and no additional packages. It is run from a terminal with `python main.py`.

## 8.2 Functions

| Function           | Purpose               | Key logic                                                                                                                    |
|--------------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------|
| readnumber(prompt) | Safe integer input    | try: int(input(prompt)); except ValueError: print message, return None                                                       |
| showmovies()       | List movies           | Loops over MOVIES.items(), unpacking (title, price), prints 'number. title - Rs. price'                                      |
| showtimings()      | List show times       | Loops over SHOW_TIMES.items() and prints 'number. time'                                                                      |
| bookticket()       | Complete booking flow | Movie check, timing check, ticket count check, total = tickets * price, summary, confirmation, append to bookings            |
| show_bookings()    | Show history          | Prints 'No bookings found.' if the list is empty; otherwise loops with enumerate(bookings, start=1)                          |
| main()             | Program entry         | Reads the name, then a while True loop that displays the menu and routes the choice with if / elif; option 5 breaks the loop |

## 8.3 Key Logic

**Safe number input.** All numeric entries use this helper, which prevents a crash when the user types text:

```
def readnumber(prompt):
    try:
        return int(input(prompt))
    except ValueError:
        print("Please enter a number.")
        return None
```

**Validation of a choice.** A result of None (bad input) or a number that is not a key in the dictionary is rejected in one check, and the function returns to the menu:

```
moviechoice=readnumber("\nEnter movie number: ")
if moviechoice not in MOVIES:
    print("Movie not available.")
    return
moviename, price=MOVIES[moviechoice]
```

**Ticket count and total.** The count must be a number greater than zero. The total is the number of tickets multiplied by the price of the chosen movie:

```
ticketcount=readnumber("Enter number of tickets: ")
if ticketcount is None or ticketcount <= 0:
    print("Please enter a valid number of tickets.")
    return
total=ticketcount * price
```

**Confirmation and saving.** The booking is appended to the list only when the answer is 'yes' (case and surrounding spaces are ignored):

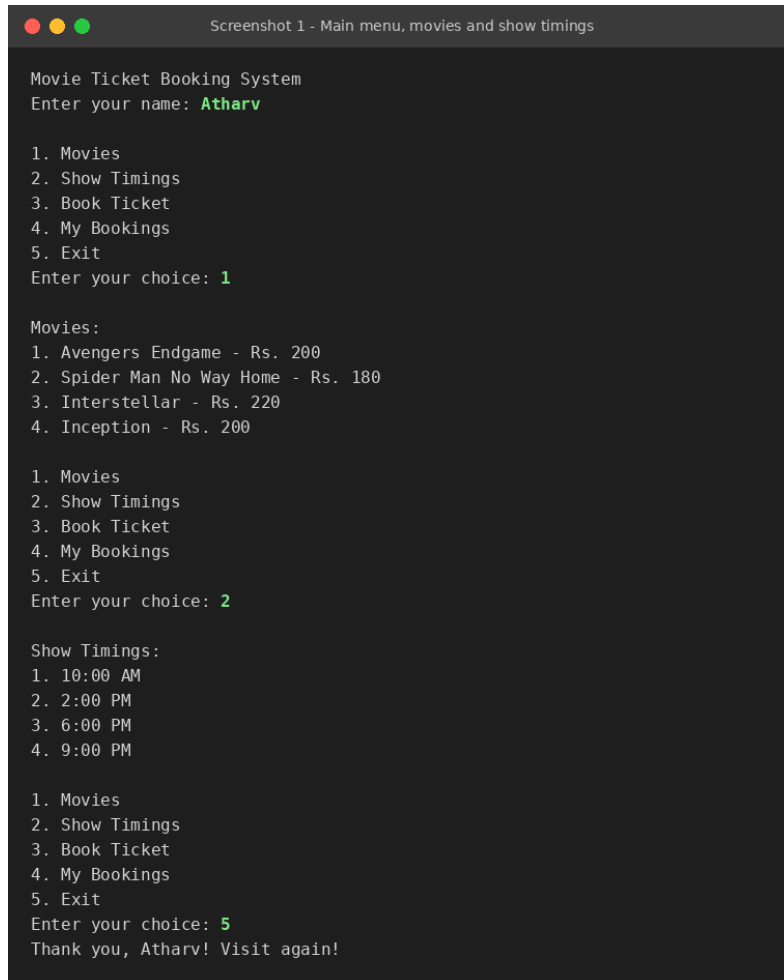
```
confirm=input("\nConfirm booking? (yes/no): ").strip().lower()
if confirm=="yes":
    bookings.append({"movie": moviename, "time": showtime,
                    "tickets": ticketcount, "total": total})
    print("Booking successful!")
else:
    print("Booking cancelled.")
```

The complete source code is included in Appendix A.

## 9. Screenshots and Results

The screenshots below are terminal captures of the program running. The values typed by the user are highlighted in green.

### 9.1 Main Menu, Movies and Show Timings



```
Movie Ticket Booking System
Enter your name: Atharv

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 1

Movies:
1. Avengers Endgame - Rs. 200
2. Spider Man No Way Home - Rs. 180
3. Interstellar - Rs. 220
4. Inception - Rs. 200

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 2

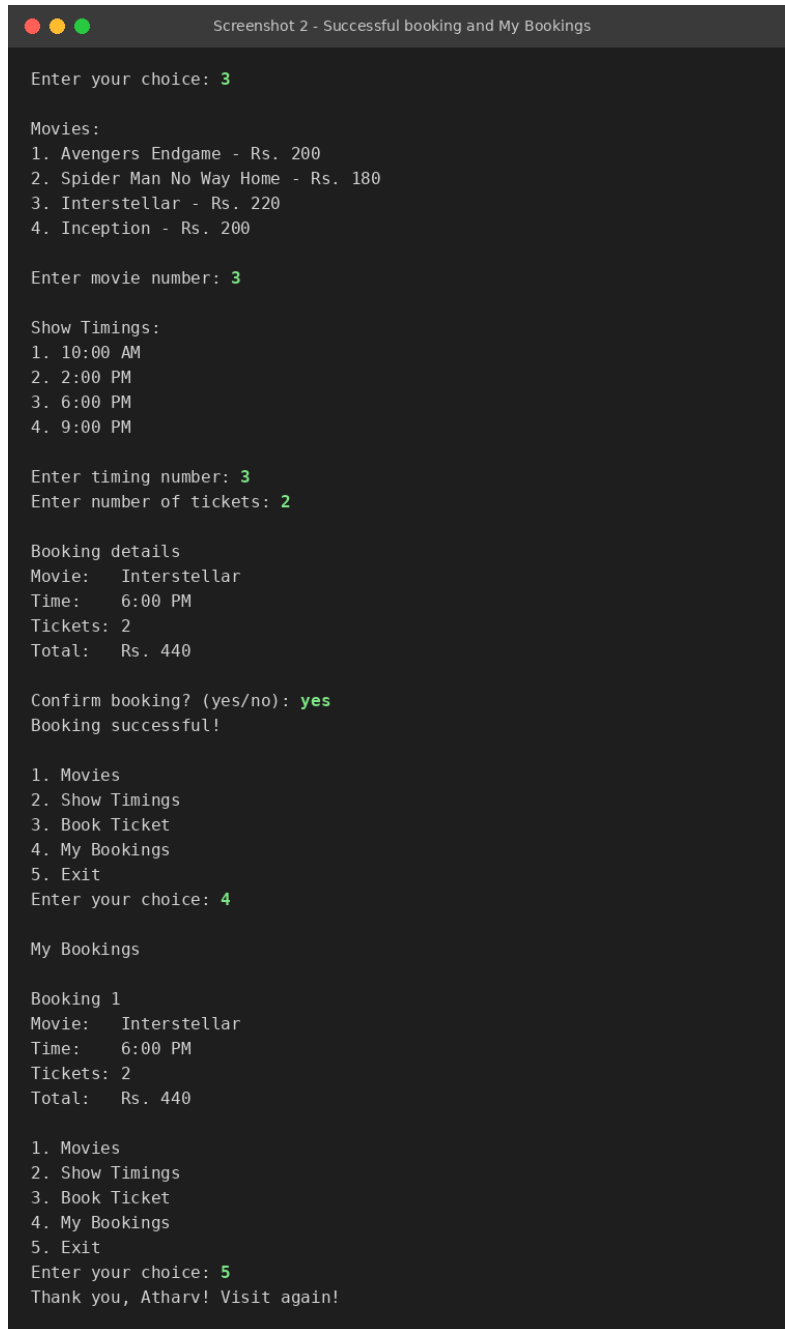
Show Timings:
1. 10:00 AM
2. 2:00 PM
3. 6:00 PM
4. 9:00 PM

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 5
Thank you, Atharv! Visit again!
```

Figure 7: Main menu, movie list and show timings (options 1 and 2)

## 9.2 Successful Booking and Booking History

Booking 2 tickets for Interstellar (Rs. 220 each) at 6:00 PM gives a total of  $2 \times 220 = \text{Rs. } 440$ . After confirmation the booking appears under My Bookings.

A screenshot of a terminal window titled "Screenshot 2 - Successful booking and My Bookings". The terminal shows a command-line interface for a movie ticket booking system. The user enters '3' to choose a movie from a list: 1. Avengers Endgame - Rs. 200, 2. Spider Man No Way Home - Rs. 180, 3. Interstellar - Rs. 220, 4. Inception - Rs. 200. Then, the user enters '3' to choose a timing from a list: 1. 10:00 AM, 2. 2:00 PM, 3. 6:00 PM, 4. 9:00 PM. Next, the user enters '2' for the number of tickets. The system displays booking details: Movie: Interstellar, Time: 6:00 PM, Tickets: 2, Total: Rs. 440. The user confirms the booking by entering 'yes'. The system then shows the main menu again. The user enters '4' to view 'My Bookings'. The system displays the booking details for 'Booking 1': Movie: Interstellar, Time: 6:00 PM, Tickets: 2, Total: Rs. 440. Finally, the user enters '5' to exit, and the system displays a thank you message: "Thank you, Atharv! Visit again!".

```
Screenshot 2 - Successful booking and My Bookings

Enter your choice: 3

Movies:
1. Avengers Endgame - Rs. 200
2. Spider Man No Way Home - Rs. 180
3. Interstellar - Rs. 220
4. Inception - Rs. 200

Enter movie number: 3

Show Timings:
1. 10:00 AM
2. 2:00 PM
3. 6:00 PM
4. 9:00 PM

Enter timing number: 3
Enter number of tickets: 2

Booking details
Movie: Interstellar
Time: 6:00 PM
Tickets: 2
Total: Rs. 440

Confirm booking? (yes/no): yes
Booking successful!

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 4

My Bookings

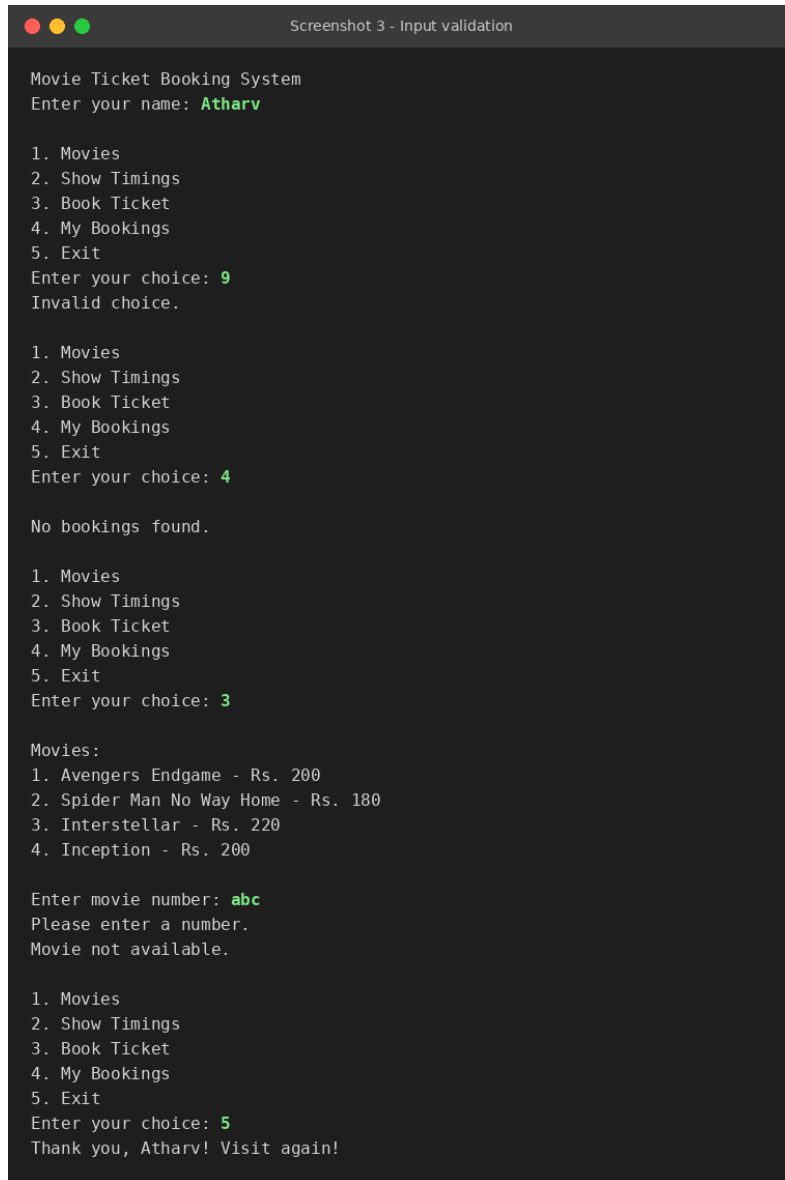
Booking 1
Movie: Interstellar
Time: 6:00 PM
Tickets: 2
Total: Rs. 440

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 5
Thank you, Atharv! Visit again!
```

Figure 8: Booking 2 tickets for Interstellar and viewing My Bookings

### 9.3 Input Validation

An invalid menu choice (9), viewing bookings before making any, and typing text (abc) instead of a movie number are all handled with clear messages and the program keeps running.



```
Movie Ticket Booking System
Enter your name: Atharv

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 9
Invalid choice.

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 4

No bookings found.

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 3

Movies:
1. Avengers Endgame - Rs. 200
2. Spider Man No Way Home - Rs. 180
3. Interstellar - Rs. 220
4. Inception - Rs. 200

Enter movie number: abc
Please enter a number.
Movie not available.

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 5
Thank you, Atharv! Visit again!
```

Figure 9: Invalid menu choice, empty booking list and non-numeric movie number

## 9.4 Cancelled Booking

When the user answers 'no' at the confirmation prompt, the booking is cancelled and nothing is saved, so My Bookings still reports 'No bookings found.'



```
Enter your choice: 3

Movies:
1. Avengers Endgame - Rs. 200
2. Spider Man No Way Home - Rs. 180
3. Interstellar - Rs. 220
4. Inception - Rs. 200

Enter movie number: 2

Show Timings:
1. 10:00 AM
2. 2:00 PM
3. 6:00 PM
4. 9:00 PM

Enter timing number: 2
Enter number of tickets: 1

Booking details
Movie: Spider Man No Way Home
Time: 2:00 PM
Tickets: 1
Total: Rs. 180

Confirm booking? (yes/no): no
Booking cancelled.

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 4

No bookings found.

1. Movies
2. Show Timings
3. Book Ticket
4. My Bookings
5. Exit
Enter your choice: 5
Thank you, Atharv! Visit again!
```

Figure 10: Cancelled booking and empty booking list

## 10. Testing Approach

### 10.1 Approach

The program was tested with **manual black-box testing**: the program is run, inputs are typed as a user would, and the output is compared with the expected result. Test cases cover normal use (valid booking, viewing lists and bookings), boundary values for the ticket count (zero and a negative number), invalid data types (text instead of numbers), unavailable movie and timing numbers, the cancel path, an empty booking list and menu handling. All 14 test cases were executed and the actual output matched the expected result.

### 10.2 Test Cases

| ID   | Test case                     | Input                                                      | Expected result                                                         | Result |
|------|-------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| TC1  | View movies                   | Menu choice 1                                              | All 4 movies listed with prices Rs. 200, 180, 220, 200                  | Pass   |
| TC2  | View timings                  | Menu choice 2                                              | Timings 10:00 AM, 2:00 PM, 6:00 PM, 9:00 PM listed                      | Pass   |
| TC3  | Valid booking                 | 3; movie 3; timing 3; tickets 2; confirm yes               | Summary shows Total Rs. 440, then 'Booking successful!'                 | Pass   |
| TC4  | View bookings after a booking | Choice 4 after TC3                                         | Booking 1: Interstellar, 6:00 PM, 2 tickets, Rs. 440                    | Pass   |
| TC5  | Invalid movie number          | 3; movie 9                                                 | 'Movie not available.' and return to menu                               | Pass   |
| TC6  | Non-numeric movie number      | 3; movie abc                                               | 'Please enter a number.' then 'Movie not available.'; no crash          | Pass   |
| TC7  | Invalid timing number         | 3; movie 1; timing 9                                       | 'Timing not available.' and return to menu                              | Pass   |
| TC8  | Zero tickets                  | 3; movie 1; timing 1; tickets 0                            | 'Please enter a valid number of tickets.'                               | Pass   |
| TC9  | Negative tickets              | 3; movie 1; timing 1; tickets -1                           | 'Please enter a valid number of tickets.'                               | Pass   |
| TC10 | Non-numeric ticket count      | 3; movie 1; timing 1; tickets abc                          | 'Please enter a number.' then 'Please enter a valid number of tickets.' | Pass   |
| TC11 | Cancel a booking              | 3; movie 2; timing 2; tickets 1; confirm no; then choice 4 | 'Booking cancelled.'; choice 4 shows 'No bookings found.'               | Pass   |
| TC12 | No bookings yet               | Choice 4 at start                                          | 'No bookings found.'                                                    | Pass   |
| TC13 | Invalid menu choice           | Menu choice 9                                              | 'Invalid choice.' and menu shown again                                  | Pass   |
| TC14 | Exit                          | Menu choice 5                                              | 'Thank you, <name>! Visit again!' and the program ends                  | Pass   |

### 10.3 Observations

- For non-numeric input (TC6, TC10) two messages appear: one from `readnumber ( )` and one from the calling `check`. This is safe and understandable, and could be merged into a single message in a future version.

- Any answer other than 'yes' at the confirmation prompt cancels the booking, which is the safe default.
- Testing is currently manual. Automated unit tests are listed under future enhancements.

## 10.4 Conclusion

All test cases passed. The program handles valid use and every invalid-input case tested without crashing, and returns the user to the menu after each error.

## 11. Challenges Faced

| Challenge                                                                                                                          | How it was addressed                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programs crash when text is typed where a number is expected, because <code>int(input())</code> raises a <code>ValueError</code> . | Wrote the reusable helper <code>readnumber()</code> that catches the error, prints a message and returns <code>None</code> ; the callers treat <code>None</code> as invalid. |
| Deciding how to store movies, timings and bookings without a database.                                                             | Used dictionaries for fixed data and a list of dictionaries for bookings, which are simple and give fast lookups.                                                            |
| <code>bookticket()</code> has many things to check (movie, timing, tickets, confirmation), which can lead to deeply nested code.   | Used early returns after each failed check so the code stays flat and each error message is specific.                                                                        |
| Printing neat, aligned output for the movie list and booking summary.                                                              | Used f-strings with consistent labels and spacing.                                                                                                                           |
| Making sure every path was tested, since there are many possible user inputs.                                                      | Prepared a table of 14 test cases covering valid, invalid, boundary and cancel scenarios and ran each one.                                                                   |

## 12. Learnings and Key Takeaways

- Dictionaries and tuples are a convenient way to store lookup data such as movies and prices.
- Functions with a single purpose make a program easier to read, test and change.
- Exception handling with `try/except` is essential for user input, and validation should happen before the data is used.
- Early returns keep validation code clean and prevent deeply nested conditions.
- A confirmation step is a simple way to prevent mistakes.
- Writing down test cases, including boundary values such as zero and negative numbers, finds problems that normal use does not.
- Documenting requirements and designing with diagrams before and after coding makes the design clearer and easier to explain.

## 13. Future Enhancements

- **Permanent storage:** save bookings to a CSV or JSON file, or a database, so they are kept after the program closes.
- **Seat selection:** show a seat layout, track available seats and prevent double booking.
- **Booking cancellation:** allow a confirmed booking to be cancelled or changed.
- **Different show times per movie:** currently every movie can be booked at every show time.
- **User accounts:** store the user's name with each booking and let returning users see their own history.
- **Modular code and automated tests:** split the program into several modules and add unit tests using Python's unittest.
- **Graphical or web interface:** add a GUI or web front end on top of the same booking logic.
- **Payments and tickets:** add payment options and a printable ticket.

## 14. References

- 1 Python Software Foundation. *The Python Tutorial* (data structures, errors and exceptions, input and output). <https://docs.python.org/3/tutorial/>
- 2 Python Software Foundation. *Python Standard Library Documentation*. <https://docs.python.org/3/library/>
- 3 van Rossum, G., Warsaw, B. and Coghlan, N. *PEP 8 - Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- 4 Object Management Group. *Unified Modeling Language (UML) Specification*. <https://www.omg.org/spec/UML/>
- 5 VITyarthi. *Build Your Own Project - General Project Instructions and Submission Guidelines*.

## Appendix A: Source Code (main.py)

```
# Movie Ticket Booking System
MOVIES={
    1: ("Avengers Endgame", 200),
    2: ("Spider Man No Way Home", 180),
    3: ("Interstellar", 220),
    4: ("Inception", 200),
}
SHOW_TIMES = {
    1: "10:00 AM",
    2: "2:00 PM",
    3: "6:00 PM",
    4: "9:00 PM",
}
bookings=[]
def readnumber(prompt):
    try:
        return int(input(prompt))
    except ValueError:
        print("Please enter a number.")
        return None
def showmovies():
    print("\nMovies:")
    for number, (title, price) in MOVIES.items():
        print(f"{number}. {title} - Rs. {price}")
def showtimings():
    print("\nShow Timings:")
    for number, showtime in SHOW_TIMES.items():
        print(f"{number}. {showtime}")
def bookticket():
    showmovies()
    moviechoice=readnumber("\nEnter movie number: ")
    if moviechoice not in MOVIES:
        print("Movie not available.")
        return
    moviename, price=MOVIES[moviechoice]
    showtimings()
    timechoice=readnumber("\nEnter timing number: ")
    if timechoice not in SHOW_TIMES:
        print("Timing not available.")
        return
    showtime=SHOW_TIMES[timechoice]
    ticketcount=readnumber("Enter number of tickets: ")
    if ticketcount is None or ticketcount <= 0:
        print("Please enter a valid number of tickets.")
        return
    total=ticketcount * price
    print("\nBooking details")
    print(f"Movie: {moviename}")
    print(f"Time: {showtime}")
    print(f"Tickets: {ticketcount}")
    print(f"Total: Rs. {total}")
    confirm=input("\nConfirm booking? (yes/no): ").strip().lower()
    if confirm=="yes":
        bookings.append({
            "movie": moviename,
            "time": showtime,
            "tickets": ticketcount,
            "total": total,
        })
        print("Booking successful!")
    else:
        print("Booking cancelled.")
def show_bookings():
    if not bookings:
        print("\nNo bookings found.")
        return
    print("\nMy Bookings")
    for number, booking in enumerate(bookings, start=1):
        print(f"\nBooking {number}")
        print(f"Movie: {booking['movie']}")
        print(f"Time: {booking['time']}")
        print(f"Tickets: {booking['tickets']}")
```

```

        print(f"Total:   Rs. {booking['total']}")
def main():
    print("Movie Ticket Booking System")
    name=input("Enter your name: ")
    while True:
        print("\n1. Movies")
        print("2. Show Timings")
        print("3. Book Ticket")
        print("4. My Bookings")
        print("5. Exit")
        choice=input("Enter your choice: ")
        if choice=="1":
            showmovies()
        elif choice=="2":
            showtimings()
        elif choice=="3":
            bookticket()
        elif choice=="4":
            show_bookings()
        elif choice=="5":
            print(f"Thank you, {name}! Visit again!")
            break
        else:
            print("Invalid choice.")
if __name__ == "__main__":
    main()

```